

video-edit-agent — User Guide

Version 0.2.2. This is a user-facing guide — for architecture and internals see `agent/core_instructions.md` and the module docstrings under `src/`.

This is the editable Markdown source for the PDF guide (`docs/video-edit-agent-user-guide.pdf`, generated by `scripts/build_user_guide.py`). Edit this file, not the PDF.

video-edit-agent

video-edit-agent is an AI video editing and production agent inside Claude Code. You talk to it in plain language — Arabic or English — and it turns raw footage, a script, or a folder of scenes into a finished, captioned video. It works fully offline with zero API keys, and is enhanced (never gated) by optional cloud providers.

Start here — install and your first request

Step 1. Open Claude Code locally.

Step 2. Give Claude the repository and ask it to install and set up the project:

```
Install and set up:  
https://github.com/eslmamin5-creator/video-edit-agent  
  
Use the recommended setup from the repository.  
Tell me if any system-level tool is missing before installing it.
```

Claude detects your OS and a compatible Python, builds a private runtime this project owns, installs the offline profile, checks `ffmpeg/Node`, and tells you when it's ready — in whichever language you're talking to it in. No API key is ever required or stored.

Step 3. Once it reports ready, just ask naturally:

Arabic: اجمع المشاهد دي · اعمل فيديو من السكريبت ده · عدل الفيديو ده

English: "Edit this video" · "Create a video from this script" · "Assemble these scenes"

You don't need any CLI command for this path. The experience is meant to feel like:

Open Claude Code → install/open video-edit-agent → talk naturally → make videos

not:

Read technical docs → memorize CLI commands → understand package internals

What can it do?

Describe what you have, and the agent routes your request — you don't need to name a workflow yourself:

You have...	You say	It runs
One raw take	"Edit this video"	Editor
A script or idea	"Create a video from this script"	Creator
Several pre-shot scenes	"Assemble these scenes"	Assembler

If your request is genuinely ambiguous, the agent asks one short clarifying question instead of guessing.

Raw video → Editor

Use Editor when you already have a single recording — talking-head, ad take, interview — that needs cleanup and polish.

It transcribes the audio, detects and cuts silence/repetition/false starts, builds an editable EDL, generates Arabic- or English-aware captions (RTL shaping, karaoke word highlighting), plans and places B-roll, applies motion graphics (including behind-subject placement where a usable mask exists), applies your Brand Profile, runs a QA pass, and renders the final video.

```
videoedit edit my_take.mp4
```

Output lands in `my_take_edit/` next to the source video. Use `--brand <name>` to apply a Brand Profile, `--offline` to force zero network calls.

Script → Creator

Use Creator when you're starting from a written script or a described idea — no footage yet.

It parses the script, plans scenes, analyzes pacing and structure, builds a storyboard, plans the assets you'll need, assembles a MasterTimeline, applies motion treatment, renders, and runs QA.

```
videoedit create my_script.txt
```

Produces a storyboard, an asset plan, a MasterTimeline, and a rendered final video, with the same inspectable-artifact pattern as Editor.

Scenes → Assembler

Use Assembler when you've already shot multiple scenes and want them combined into one film.

It discovers the scenes, preserves your intended order (or the order you ask for), builds a fast rough cut, then plans continuity and finishes with real transitions, audio crossfade, and loudness normalization — reusing the rough cut's plan rather than starting over.

```
videoedit assemble my_scenes_dir --rough  
videoedit assemble my_scenes_dir --finish
```

`--rough` builds a fast preview preserving your scene order. `--finish` reuses that plan to add real transitions, audio crossfade, and loudness normalization for the final film.

Arabic-first experience

Talking to it in Arabic is a product benefit, not just a technical capability:

- You talk to Claude in Arabic and get Arabic back — naturally, not translated from an English template.
- Captions display correctly: RTL, properly shaped, karaoke-style word highlighting, and mixed Arabic/English on the same line.

- Egyptian, Gulf, Saudi, and MSA dialects are preserved exactly as spoken. The agent never silently turns dialect into formal Arabic.
- Interaction language never forces transcript translation — the language you use to talk to Claude is independent of your video's own language.

Underneath, the single most important rule in this project is: **the agent transcribes exactly what was said and never translates, rewrites, formalizes, or dialect-converts spoken Arabic.** Source audio has final authority. Mechanically this means RTL shaping and bidi reordering for display, Arabic-aware glyph joining so letters connect correctly, and a dialect guard that blocks silent normalization to MSA — details a developer might want, not something you need to know to use the product.

Motion, B-roll, and Brand Profiles

Motion — a built-in simple motion engine that always works offline; optional Remotion rendering for richer motion (needs Node/npm, still no API key); automatic engine fallback that's logged, never silent; and behind-subject compositing status reporting.

B-roll — planning with automatic provider fallback; a fully offline path using a local library; optional cloud enhancement (Gemini image/video generation) when API keys are set.

Brand Profiles — reusable colors, fonts, and caption presets per brand, stored as plain folders under `brands/<name>/`. No source changes needed to add one:

```
videoedit brand init my_brand
videoedit brand validate my_brand
videoedit edit my_take.mp4 --brand my_brand
```

What happens after you ask?

Editor: 1. You give Claude the video. 2. It checks that local tools are ready. 3. It transcribes the speech. 4. It cleans obvious silence/repetition/false starts. 5. It prepares captions and visual treatment. 6. It plans motion/B-roll where useful. 7. It renders the video. 8. It checks the result and gives you the final files.

Creator: 1. You give a script or idea. 2. The agent understands its structure. 3. It splits the idea into scenes. 4. It builds a storyboard and asset plan. 5. It produces a timeline and motion treatment. 6. It renders and checks the result.

Assembler: 1. You give a folder of scenes. 2. The agent discovers the scenes. 3. It preserves (or follows your requested) order. 4. It makes a rough cut. 5. You can review and adjust. 6. It finishes transitions and audio. 7. It delivers the final film.

What you receive

You get the final video, plus the files that explain what the agent changed, so you can review the decisions or continue the project later:

- **The final video** — `final.mp4`.
- **Captions and transcript**, where applicable — `captions.ass`, `transcript_unified.json`.

- **Editable project decisions** — `edl.json` (the edit decision list) and the MasterTimeline for Creator, both re-renderable without redoing the whole pipeline.
- **Transparency reports** — `project.md` (why the agent made each decision) and `motion/B-roll/QA` plan JSON files.

Every stage is a plain file, never hidden state — you can open any of them directly.

Offline vs. Cloud

You can use the core workflows — Editor, Creator, and Assembler — with zero API keys.

Capability	Works with zero API keys	What cloud adds
Transcription	Yes — local Faster-Whisper	Gemini / ElevenLabs transcription (optional, needs an API key)
Captions (RTL, shaping, karaoke highlight)	Yes, fully local	—
Editing (silence/repetition/false-start, EDL)	Yes, fully local	—
Motion graphics	Yes — built-in simple engine	Remotion (needs Node/npm, still no API key) for richer motion
B-roll	Yes — local library/planning	Gemini image/video generation for AI-generated B-roll
Visual QA	Yes — basic local checks	Gemini vision for richer QA
Brand Profiles, EDL, render, Assembler rough/finish	Yes, fully local	—

GEMINI_API_KEY and ELEVENLABS_API_KEY are read only from environment variables, never from a config file, and never printed or logged by any command — including `videoedit doctor`.

How to ask for changes

The conversation doesn't stop at the first result. Because every stage is a re-renderable, inspectable decision — not a baked video you'd have to redo from scratch — the agent can apply a change and re-render just the affected part.

Arabic: هنا · خلي المشهد ده أقصر · حافظ على اللهجة المصرية زي ما هي · الكابشن كبير زيادة · خلي الفيديو أسرع · شيل الجملة دي · كمل من آخر نقطة · اشتغل من غير Cloud · قلل الحركة

English: "Remove this line" · "Make the video faster" · "The captions are too big" · "Keep the Egyptian dialect as it is" · "Make this scene shorter" · "Reduce the motion here" · "Work without cloud" · "Continue from where we left off"

Troubleshooting

Caption text is wrong. Tell Claude the correct word and ask it to update the caption.

The video is too long. Ask it to shorten the video or remove a specific part.

You want offline only. Tell it: "Work offline only" / اشتغل Offline فقط .

A system tool is missing. Claude tells you exactly which system-level tool (like `ffmpeg`) is missing and asks before installing or changing anything on your system — it never installs OS-level software silently.

The work stopped partway. Ask Claude to inspect the project state and continue from the saved project. Every stage is a plain file it can re-read, so it can pick up supported workflows without starting over — though not every interruption is guaranteed to resume cleanly; ask it to check first.

For a live, machine-specific readiness view at any time:

```
videoedit doctor
```

Prints the full capability matrix (`ffmpeg`, `Node`, providers, engines) plus the last `videoedit setup` run's summary. Run `videoedit setup --check` for a read-only readiness check, or `videoedit setup --repair` to force-rebuild the private runtime from scratch without touching anything outside `.runtime/`.

Known limitations

- Arabic ASR on real spoken audio and Remotion motion rendering are not yet verified against real acceptance runs in this environment — see the capability status table in the README for the full honesty breakdown.
- J-Cut/L-Cut audio-lead edits are conceptual only, not implemented.
- HyperFrames is not a real installable motion engine; the router always falls back to a working engine and logs the fallback — never a silent substitution.
- Resuming after an interruption is supported for Assembler's rough → finish flow; other workflows do not yet guarantee a clean resume from every interruption point.

Advanced / technical appendix

Install with Claude Code (full detail)

1. Clone or install this repository into your Claude Code skills directory.
2. Open (or restart) Claude Code.
3. Say: **"Set up video-edit-agent and verify it."**

Claude bootstraps everything: detects your OS and a compatible Python (prefers 3.11), builds a private runtime this project owns (`.runtime/venv` — never your system Python), installs the offline profile, checks `ffmpeg/ffprobe` and `Node/npm`, and reports readiness in plain language, in whichever language you're talking to it in. It never installs OS-level software silently — if `ffmpeg` is missing, it gives you the exact command for your OS and waits for you to run it.

Source install (power users)

```
git clone https://github.com/eslmamin5-creator/video-edit-agent
cd video-edit-agent
python3 -m pip install -e . # base CLI only
videoedit setup --profile full-local # builds .runtime/venv, installs the offline profile
```

`videoedit setup` supports `--profile local|subject|motion|core` for a narrower install, `--check` for a read-only readiness check, and `--repair` to force-rebuild the private runtime.

CLI command reference

Command	Purpose
<code>videoedit edit <video></code>	Full pipeline: transcribe → cut → caption → B-roll → motion → render → QA
<code>videoedit create <script></code>	Script → storyboard → asset plan → motion → render → QA
<code>videoedit assemble <scenes_dir> --rough</code>	Fast rough cut from a folder of pre-shot scenes
<code>videoedit assemble <scenes_dir> --finish</code>	Finish with real transitions + loudness normalization
<code>videoedit doctor</code>	Print the capability matrix
<code>videoedit providers</code>	List transcription/motion providers and availability
<code>videoedit setup [--profile ...] [--check] [--repair]</code>	Bootstrap the private runtime, check readiness
<code>videoedit config show / set</code>	Inspect or edit layered config (never secrets)
<code>videoedit brand init / validate</code>	Create or validate a Brand Profile
<code>videoedit project inspect <dir></code>	Print a project's <code>project.md</code> memory

Project artifacts (full list)

`project.md`, `transcript_unified.json`, `edl.json`, `captions.ass`, motion plan JSON, B-roll plan JSON, QA plan JSON, and `final.mp4`.